



Sophos Central (Fusion) Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-20

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	1
Discovering the tenant routing values (required)	2
Mapping values into Crogl.....	2
Verifying the setup.....	3

Overview

The Sophos Central connector authenticates to the tenant-scoped Sophos Central REST API using OAuth2 client credentials. A Sophos Central administrator creates an API credential, then runs a single `whoami` call to discover the two tenant-routing values the connector needs (the data-region API host and the tenant ID). These steps are performed by a customer-side Sophos Central administrator — not by Crogl.

NOTE

Sophos Central and Sophos Fusion are the same platform (Fusion is the console rebrand on the Sophos Central API). This is **not** Secureworks Taegis — a separate platform with separate credentials. A Sophos Central credential does not authenticate to Taegis.

What kind of account to create

Sophos Central uses an **API credential** (a machine client), not a named user login. In Sophos Central Admin, go to **Global Settings** → **API Credentials Management**, click **Add Credential**, give it a name (e.g. `crogl-connector`), and select a role. This yields a `client_id` and `client_secret`.

For a Sophos Central Partner or Enterprise (multi-tenant) account the menu path and the resulting token scope differ; this guide covers the single-tenant (Admin) case. Multi-tenant provisioning is (TBD — verify against a partner console).

Permissions to grant

Assign the API credential a role with **read** access to the surfaces the connector uses: alerts (Common), cases, and endpoints. The connector performs read-only GETs; it never needs management/remediation permissions. Grant the least-privilege read role available (e.g. a "Read-Only" service-principal role). The exact role name is (TBD — verify against the live console); note that Sophos API credentials are role-scoped, so a too-narrow role yields `403` on some resources even though the token mints fine.

Generating the credential

Sophos Central shows the `client_secret` at creation. Copy it immediately and store it securely — it may not be retrievable later; if lost, regenerate the credential. The `client_id` remains visible on the credential afterward.

Access tokens minted from these credentials are short-lived; Crogl re-mints them automatically, so no manual refresh is required.

Discovering the tenant routing values (required)

The connector needs the tenant's data-region host and tenant ID. Obtain both with one call after minting a token:

```
# 1. Mint a token
TOKEN=$(curl -s -X POST https://id.sophos.com/api/v2/oauth2/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d
"grant_type=client_credentials&client_id=$CLIENT_ID&client_secret=$CLIENT_SECRET&scope
=token" \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# 2. whoami -> tenant id + data-region host
curl -s https://api.central.sophos.com/whoami/v1 -H "Authorization: Bearer $TOKEN"
```

The response looks like:

```
{
  "id": "<tenant-uuid>",
  "idType": "tenant",
  "apiHosts": { "global": "https://api.central.sophos.com",
                "dataRegion": "https://api-us01.central.sophos.com" }
}
```

- `id` → the connector's `tenant_id`.
- `apiHosts.dataRegion` (host only, no scheme) → the connector's `api_host`.

Mapping values into Crogl

One row per `community/sophos-central/connector.json` credential-schema field:

Crogl field	Where it came from	Notes
<code>client_id</code>	The created API credential's client ID	Visible on the credential after creation.
<code>client_secret</code>	The secret shown once at creation	Copy at creation; regenerate if lost. Stored encrypted by Crogl.
<code>api_host</code>	<code>apiHosts.dataRegion</code> from <code>whoami</code>	Host only, no scheme (e.g. <code>api-us01.central.sophos.com</code>). All data calls must go to this region host.

Crogl field	Where it came from	Notes
<code>tenant_id</code>	<code>id</code> from <code>whoami</code>	Sent as the <code>X-Tenant-ID</code> header on every call. Entered as a secret field so Crogl stores it encrypted — it is an identifier, not a password, but Crogl's header-injection binding requires it to be a stored (secret) field.

Verifying the setup

Confirm the credential works before creating the connector:

- The token mint (step 1 above) returns `200` with an `access_token` — the `client_id/client_secret` are valid.
- `whoami` (step 2) returns `200` with your tenant `id` and `dataRegion`.
- A tenant-scoped read confirms routing + role:

```
curl -s -o /dev/null -w "%{http_code}\n" \
  "https://<api_host>/endpoint/v1/endpoints?pageSize=1" \
  -H "Authorization: Bearer $TOKEN" -H "X-Tenant-ID: <tenant_id>"
```

- `200` — routing + role are correct (an empty `items` array is normal on a tenant with no enrolled endpoints).
- `401` — token invalid/expired (bad credential).
- `403` — credential is valid but its role lacks access to that resource (widen the role).
- `404 "Unable to identify proxy for host"` — wrong `api_host`; use the exact `dataRegion` host from `whoami`.

After creating the connector instance in Crogl, verify end-to-end with a bounded query (e.g. endpoints, `--limit 1`). A fresh trial tenant with no Sophos agent deployed legitimately returns zero rows; deploy at least one endpoint agent to generate alert/case/endpoint data for a meaningful test.